

Web Application Vulnerability Scanner

Duration: 08\07\2025- 12\07\2025

Submitted By: AMIT MAURYA

Mentor/Guide: Elevate Labs

Tools & Technologies: Python, Flask, requests, BeautifulSoup, Regex, OWASP Top 10

1. Introduction

Web applications are common targets for cyberattacks due to their exposure to the internet and user interaction. This project aims to build an automated web application vulnerability scanner that detects common vulnerabilities listed in the OWASP Top 10, including Cross-Site Scripting (XSS), SQL Injection (SQLi), and Cross-Site Request Forgery (CSRF). The scanner provides a user-friendly interface for scanning and reporting vulnerabilities, assisting developers and security analysts in securing their web applications.

2. Abstract

This project implements a Python-based security scanner that analyzes web applications for injection points, inspects forms and inputs, and tests them with known attack payloads. By leveraging libraries like requests and BeautifulSoup, the scanner crawls URLs, injects malicious inputs, and monitors server responses for signs of vulnerabilities. A Flask-based web interface is provided to initiate scans, view results, and download reports. Each identified issue is logged with evidence, severity levels, and timestamps to aid in remediation.

3. Tools Used

- Python: Core programming language for backend logic.
- Flask: Lightweight web framework used for the GUI and server management.
- requests: Used for HTTP requests to target web applications.
- BeautifulSoup: Used to parse and analyze HTML for input fields and forms.
- Regex: Used to detect reflected payloads and error messages indicating vulnerabilities.
- OWASP Top 10: Industry-standard checklist for vulnerability types and severity.

4. Steps Involved in Building the Project

1. Requirement Analysis: Identified key features required for the scanner based on the OWASP Top 10.
2. Crawler Development: Implemented a URL crawler using requests and BeautifulSoup to extract all forms and input fields.
3. Payload Injection Engine: Designed a system to inject malicious payloads for vulnerabilities like XSS and SQLi.

4. Response Analysis Module: Used pattern matching and regular expressions to detect vulnerable responses.
5. Flask-based UI: Developed a user interface for users to input the target URL, start/stop scans, and view results.
6. Logging and Reporting: Vulnerabilities were logged with the URL, payload used, response snippet, and severity.

5. Conclusion

The project successfully delivered a functional web application vulnerability scanner with automated payload injection, response analysis, and web interface. It helps users detect major security issues such as XSS and SQLi efficiently. This tool can be extended further to include authentication handling, deeper crawling, and integration with advanced vulnerability databases. It serves as a foundational step toward building more comprehensive penetration testing tools.